

开源软件基础

Python 基础

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 16, 2017



Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

为什么选择 Python

- 流行
- 简单 (?)
- 丰富的生态系统
- 交互界面友好

Python 版本

- Python 存在两个用途广泛的版本：2.x 和 3.x
- 两者不完全兼容
- Python 2.x 分支版本在 2020 年将停止更新
- Python 3 对 UTF-8 支持更好
- 很多公司/开源社区仍默认选择 Python 2

Python 发行版

- vanilla Python
- Anaconda
- Python (x,y)
- Active Python

Jupyter Notebook

 **jupyter** Untitled3 Last Checkpoint: a minute ago (unsaved changes)  [Logout](#)

File Edit View Insert Cell Kernel Help Trusted  Python 3 

          Code  

```
In [1]: 1 + 1
Out[1]: 2

In [2]: print("hello world")
hello world

In [3]: def fib(n):
        if n <= 1:
            return 1
        else:
            return fib(n - 1) + fib(n - 2)

In [4]: fib(10)
Out[4]: 89

In [ ]: 
```

本学期将涉及的 Python 知识

知识点

- 流程控制（结构化）
- 面向对象（皮毛）
- 数据获取
- 数据存储
- 数据可视化
- Web 开发（皮毛）

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- 4 函数
- 5 递归

第一个 Python 程序

```
$ipython3
Python 3.5.2+ (default, Sep 22 2016, 12:18:14)
Type ``copyright``, ``credits`` or ``license`` for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?                -> Introduction and overview of IPython's features.
%quickref        -> Quick reference.
help             -> Python's own help system.
object?         -> Details about 'object', use 'object??' for extra details.

In [1]:
```

第一个 Python 程序

```
In [1]: print("Hello world")  
Hello world
```

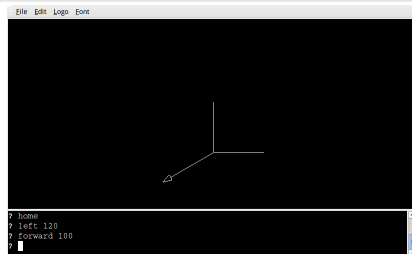
```
In [2]: print(1 + 2)  
3
```

```
In [3]:
```

Python Turtle

Logo

Python Turtle 受 Logo 语言启发而来。



Python Turtle

```
to tree :len :n
  if :n < 1 [stop]
  forward :len
  left 45
  tree (:len / 2) (:n - 1)
  right 90
  tree (:len / 2) (:n - 1)
  left 45
  back :len
end
```

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

数值

- Python 解释器可以当计算器使
- 加减乘除都支持
- 括号也能用
- 整形类型为 int
- 有小数点的类型为 float

数值

```
>>> 2 + 2
```

```
4
```

```
>>> 50 - 5*6
```

```
20
```

```
>>> (50 - 5*6) / 4
```

```
5.0
```

```
>>> 8 / 5 # 除法返回浮点数
```

```
1.6
```


数值

- 除法/返回浮点型
- 整数除法用//
- 求模用%
- 幂运算用 **
- 赋值用 =
- 不赋值直接用报错
- 混用整数浮点数类型提升

数值

```
>>> 17 / 3 # classic division returns a float
5.666666666666667
>>>
>>> 17 // 3 # floor division discards the fractional
5
>>> 17 % 3 # the % operator returns the remainder of
2
>>> 5 * 3 + 2 # result * divisor + remainder
17
```

数值

```
>>> 5 ** 2 # 5 squared
```

```
25
```

```
>>> 2 ** 7 # 2 to the power of 7
```

```
128
```

数值

```
>>> width = 20  
>>> height = 5 * 9  
>>> width * height  
900
```

数值

```
>>> n # try to access an undefined variable
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

数值

```
>>> 4 * 3.75 - 1  
14.0
```

数值

- 交互模式下 `_` 是特殊变量
- 上一次计算的结果
- 类似 bash 的 `$?` 或 cmd 的 `%errorlevel%`
- 做计算器很方便
- 非交互模式不能用
- 默认交互环境（非 ipython）下赋值会覆盖
- 除了整数、浮点数，也支持复数

数值

```
>>> tax = 12.5 / 100
>>> price = 100.50
>>> price * tax
12.5625
>>> price + _
113.0625
>>> round(_, 2)
113.06
```


数值

```
$ cat b.py
#!/usr/bin/python3
def foo():
    return 3
```

```
foo()
print(_)
```

```
$ python3 b.py
Traceback (most recent call last):
  File ``b.py'', line 7, in <module>
    print(_)
NameError: name '_' is not defined
```

Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- 4 函数
- 5 递归

字符串

- 除了数值，Python 同样支持字符串
- 可以用单引号
- 也可以用双引号
- 没有区别
- \ 用于转义
- 用 r 省去转义 (raw)
- """...""" 或 '''...''' 支持换行
- \ 忽略回车

字符串

```
>>> 'spam eggs'    # single quotes
'spam eggs'
>>> 'doesn\'t'     # use \' to escape the single quote...
"doesn't"
>>> "doesn't"      # ...or use double quotes instead
"doesn't"
>>> '"Yes," he said.'
'"Yes," he said.'
>>> "\"Yes,\" he said."
'"Yes," he said.'
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
```

字符串

```
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
>>> print('"Isn\'t," she said.')
"Isn't," she said.
>>> s = 'First line.\nSecond line.' # \n means newline
>>> s # without print(), \n is included in the output
'First line.\nSecond line.'
>>> print(s) # with print(), \n produces a new line
First line.
Second line.
```

字符串

```
>>> print('C:\some\name')    # here \n means newline!  
C:\some  
ame  
>>> print(r'C:\some\name')   # note the r before the qu  
C:\some\name
```

字符串

```
print("""\
Usage: thingy [OPTIONS]
    -h                Display this usage mess
    -H hostname       Hostname to connect to
""")
```

Display this usage mess
Hostname to connect to

字符串

- 加号拼接字符串
- 乘号重复字符串
- 字符串常量直接拼接
- 字符串可下标
- 0 起始, -1 最后
- : 用于切片, 左闭右开

字符串

```
>>> # 3 times 'un', followed by 'ium'
>>> 3 * 'un' + 'ium'
'unununium'
```

字符串

```
>>> 'Py' 'thon'  
'Python'
```

字符串

```
>>> prefix = 'Py'
>>> prefix 'thon'    # can't concatenate a variable and
...
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
...
SyntaxError: invalid syntax
```

字符串

```
>>> prefix + 'thon'  
'Python'
```

字符串

```
>>> text = ('Put several strings within parentheses '  
...        'to have them joined together.')
```

>>> text

'Put several strings within parentheses to have them j

字符串

```
>>> word = 'Python'
>>> word[0]    # character in position 0
'P'
>>> word[5]    # character in position 5
'n'
```

字符串

```
>>> word[-1]    # last character
'n'
>>> word[-2]    # second-last character
'o'
>>> word[-6]
'p'
```

字符串

```
>>> word[0:2]    # characters from position 0 (included)
'Py'
>>> word[2:5]    # characters from position 2 (included)
'tho'
```


字符串

```
>>> word[:2] + word[2:]  
'Python'  
>>> word[:4] + word[4:]  
'Python'
```

字符串

```
>>> word[:2]    # character from the beginning to posit
'Py'
>>> word[4:]    # characters from position 4 (included)
'on'
>>> word[-2:]   # characters from the second-last (incl
'on'
```

字符串

+	-	-	-	+	-	-	-	+	-	-	-	+	-	-	-	+
	P		y		t		h		o		n					
+	-	-	-	+	-	-	-	+	-	-	-	+	-	-	-	+
0	1	2	3	4	5	6										
-6	-5	-4	-3	-2	-1											

字符串

```
>>> word[42] # the word only has 6 characters
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

字符串

```
>>> word[4:42]  
'on'  
>>> word[42:]  
,,
```

字符串

- Python 字符串是不可变的，与 Java 类似
- len 返回长度，类比 strlen

字符串

```
>>> word[0] = 'J'
```

```
...
```

```
TypeError: 'str' object does not support item assignment
```

```
>>> word[2:] = 'py'
```

```
...
```

```
TypeError: 'str' object does not support item assignment
```

字符串

```
>>> 'J' + word[1:]  
'Jython'  
>>> word[:2] + 'py'  
'Pypy'
```


字符串

The built-in function `len` returns the length of a string:

字符串

```
>>> s = 'supercalifragilisticexpialidocious'  
>>> len(s)  
34
```

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

列表

- Python 支持若干数据类型
- 最常见的是列表
- 与数组类似
- 允许元素类型不同
- 和字符串类似，可下标可切片
- 加法和乘法也和字符串类似
- len 返回长度
- 与字符串不同的，列表是可变的
- append 方法用于追加
- 多维数组用嵌套列表实现

列表

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]
```

列表

```
>>> squares[0]    # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:]  # slicing returns a new list
[9, 16, 25]
```

列表

```
>>> squares[:]  
[1, 4, 9, 16, 25]
```

列表

```
>>> squares + [36, 49, 64, 81, 100]  
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```


列表

```
>>> cubes = [1, 8, 27, 65, 125] # something's wrong h
>>> 4 ** 3 # the cube of 4 is 64, not 65!
64
>>> cubes[3] = 64 # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
```

列表

```
>>> cubes.append(216) # add the cube of 6
>>> cubes.append(7 ** 3) # and the cube of 7
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

列表

```
>>> letters = ['a', 'b', 'c', 'd']  
>>> len(letters)  
4
```

列表

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

流程控制初步

Of course, we can use Python for more complicated tasks than adding two and two together. For instance, we can write an initial sub-sequence of the Fibonacci series as follows:

流程控制初步

```
>>> # Fibonacci series:
... # the sum of two elements defines the next
... a, b = 0, 1
>>> while b < 10:
...     print(b)
...     a, b = b, a+b
...
1
1
2
3
5
8
```

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

Outline

- ① Python 环境安装
- ② Hello world
- ③ 变量、类型、基本流程控制
 - 数值
 - 字符串
 - 列表
- ④ 函数
- ⑤ 递归

递归

To understand recursion, first you have to understand recursion.

阶乘

```
def factorial(n):  
    if n <= 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

递归

With a little help from my friends.

– The Beatles

分形

分形 (Fractal), 又称碎形、残形, 通常被定义为「一个粗糙或零碎的几何形状, 可以分成数个部分, 且每一部分都 (至少近似地) 是整体缩小后的形状」, 即具有自相似的性质。¹

¹<https://zh.wikipedia.org/wiki/>

Turtle

```
import turtle

def tree(length,n):
    """ paints a branch of a tree """
    if length < (length/n):
        return
    turtle.forward(length)
    turtle.left(45)
    tree(length * 0.5,length/n)
    turtle.right(90)
    tree(length * 0.5,length/n)
    turtle.left(45)
    turtle.backward(length)
    return
```

边界条件
主干
左转分叉
左子树
右转分叉
右子树
转回
回到起点
返回



Turtle

